

LAPORAN PRAKTIKUM PEKAN 09
PEMROGRAMAN WEB
CODEIGNITER 3 + HMVC + REST API +
CRUD AJAX



Telkom
University

FUTUWAH IQAMATUL HAQI

707012500099

49-04

1. Tujuan Praktikum

- Memahami cara membangun RESTful API menggunakan Framework CodeIgniter 3.
- Menerapkan konsep Authentication (Register, Login, Logout, dan pengecekan user dengan Bearer Token).
- Mengimplementasikan operasi CRUD (Create, Read, Update, Delete) pada resource Post beserta fitur upload gambar.
- Memahami penggunaan Postman untuk pengujian API.
- Melatih kemampuan konfigurasi database, routing, dan penanganan error pada CodeIgniter 3.

2. Alat & Bahan

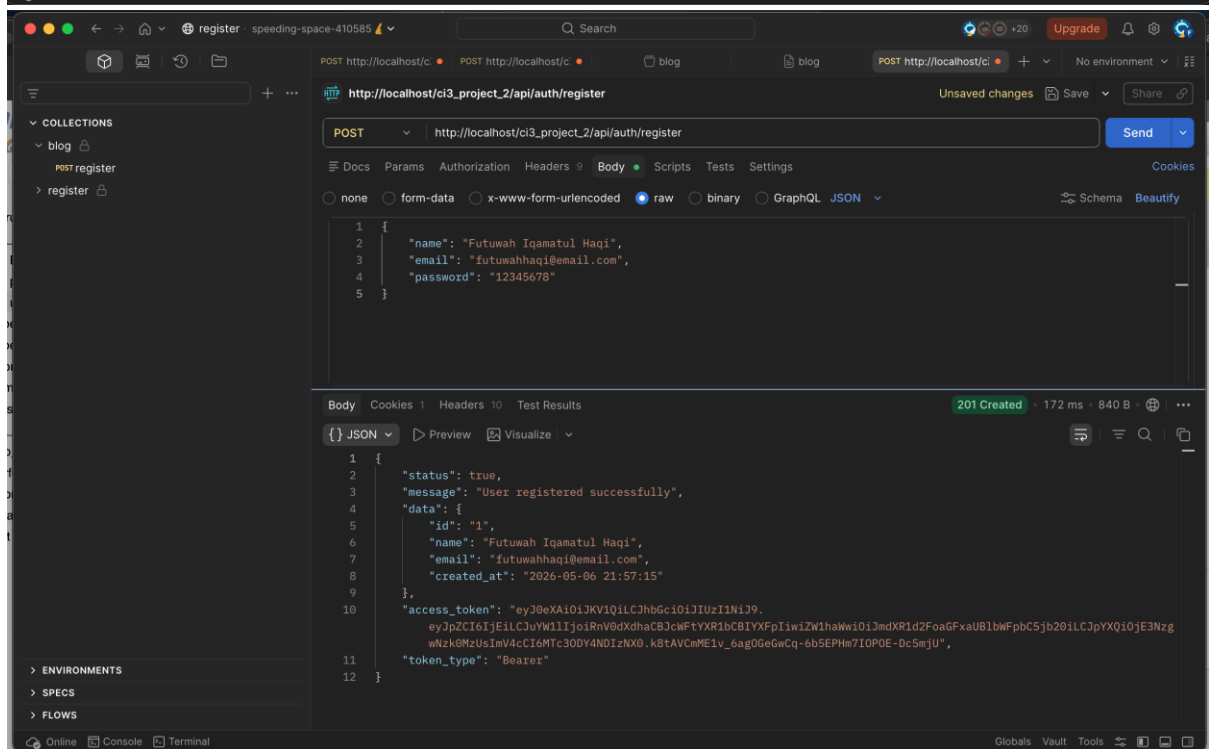
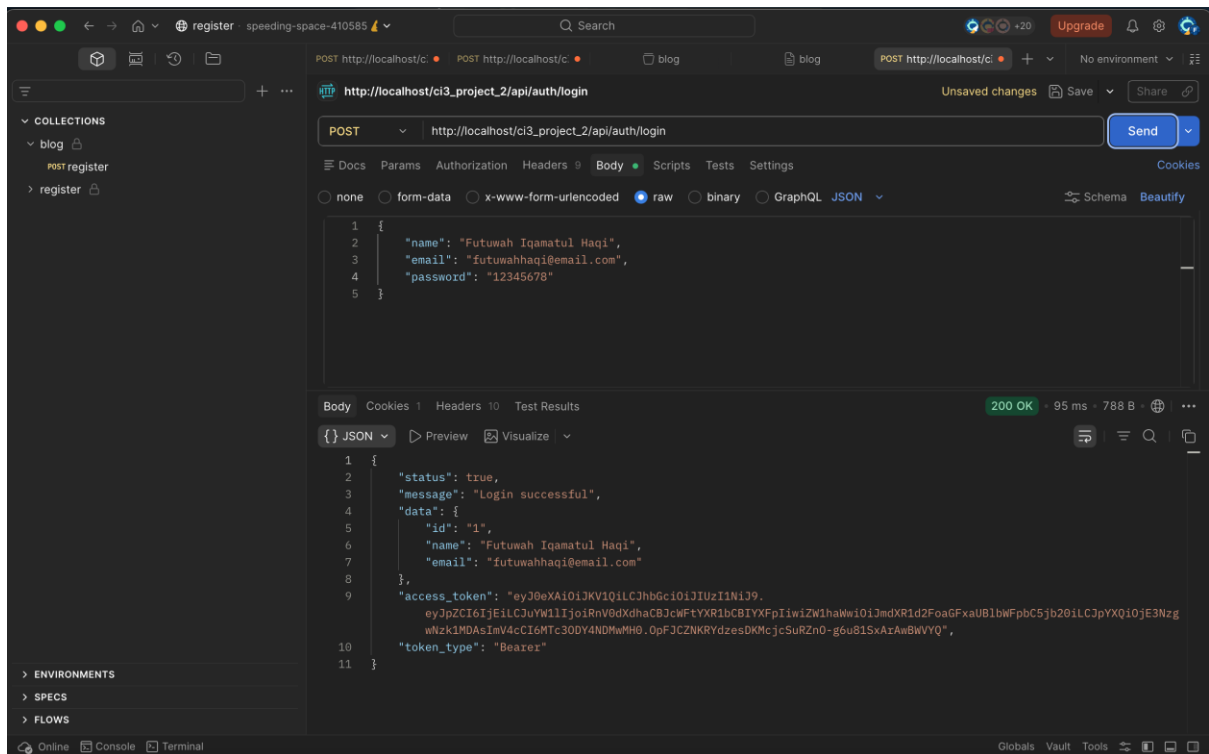
- Laptop MacBook Air
- XAMPP (Apache + MySQL + PHP)
- Postman (API Testing Tool)
- Visual Studio Code / Text Editor
- Browser (Google Chrome / Safari)

3. Langkah-Langkah Praktikum

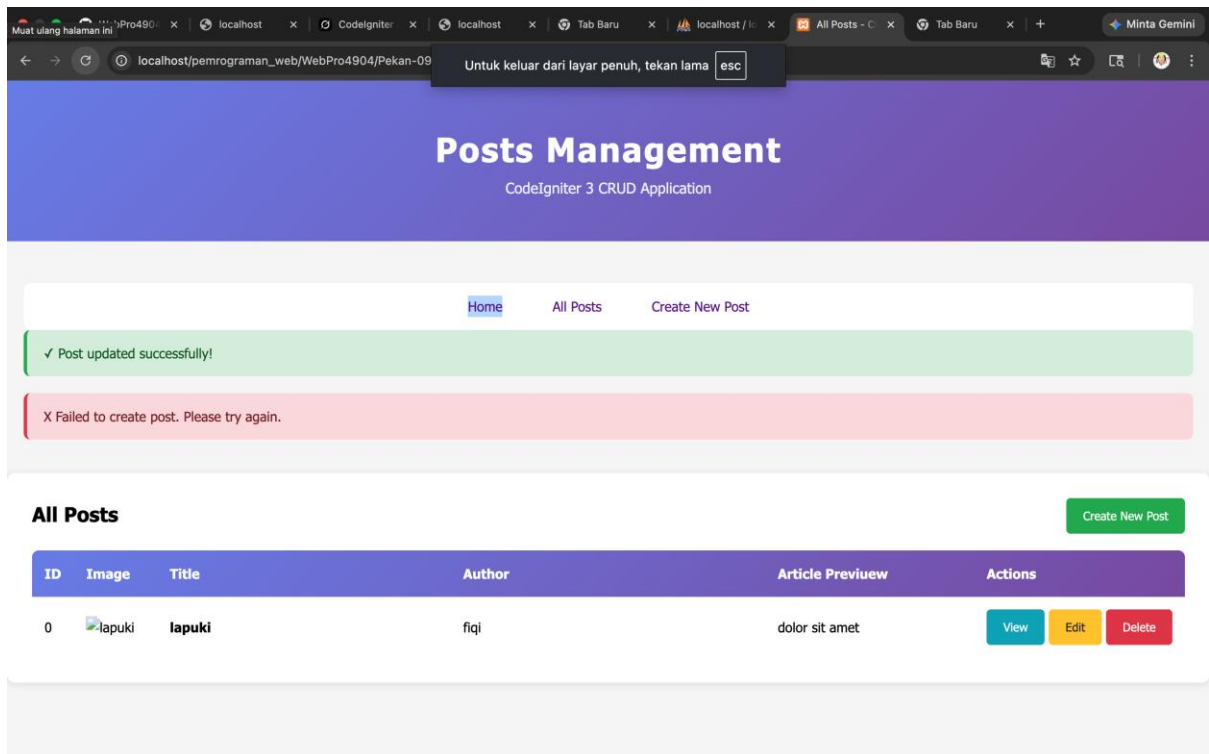
- Install dan konfigurasi XAMPP di MacBook.
- Menempatkan folder ci3_project ke dalam htdocs.
- Membuat database di phpMyAdmin.
- Konfigurasi config.php, database.php, dan index.php.
- Membuat Controller untuk Auth dan Post.
- Membuat Model dan mengatur routing.
- Pengujian API menggunakan Postman:
 - Register
 - Login + mendapatkan Token
 - CRUD Post (dengan upload gambar)

4. Hasil & Dokumentasi

Hasil di Postman



Hasil di web



Kode

Auth.php

```
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Auth extends MX_Controller {

    public function __construct()
    {
        parent::__construct();
        $this->load->model('User_model');
        $this->load->library('jwt');
        $this->load->library('form_validation');
    }

    public function register()
    {
        header('Content-Type: application/json; charset=utf-8');
        ob_clean();

        try {
            $input = json_decode(file_get_contents('php://input'), true);
```

```
if (empty($input['name']) || empty($input['email']) || empty($input['password'])) {
    http_response_code(400);
    echo json_encode([
        'status' => false,
        'message' => 'Name, email, and password are required'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

if (!filter_var($input['email'], FILTER_VALIDATE_EMAIL)) {
    http_response_code(400);
    echo json_encode([
        'status' => false,
        'message' => 'Invalid email format'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

if ($this->User_model->email_exists($input['email'])) {
    http_response_code(409);
    echo json_encode([
        'status' => false,
        'message' => 'Email already registered'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

if (strlen($input['password']) < 6) {
    http_response_code(400);
    echo json_encode([
        'status' => false,
        'message' => 'Password must be at least 6 characters'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

$user_data = [
    'name' => htmlspecialchars($input['name']),
    'email' => strtolower($input['email']),
    'password' => password_hash($input['password'], PASSWORD_BCRYPT)
];
```

```

$user_id = $this->User_model->create($user_data);

if (!$user_id) {
    http_response_code(500);
    echo json_encode([
        'status' => false,
        'message' => 'Failed to create user'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

$user = $this->User_model->get_by_id($user_id);
$token = $this->jwt->create([
    'id' => $user->id,
    'name' => $user->name,
    'email' => $user->email
]);

http_response_code(201);
echo json_encode([
    'status' => true,
    'message' => 'User registered successfully',
    'data' => [
        'id' => $user->id,
        'name' => $user->name,
        'email' => $user->email,
        'created_at' => $user->created_at
    ],
    'access_token' => $token,
    'token_type' => 'Bearer'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
    http_response_code(500);
    echo json_encode([
        'status' => false,
        'message' => 'Server error: ' . $e->getMessage()
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}
}

public function login()

```

```
{
header('Content-Type: application/json; charset=utf-8');
ob_clean();

try {
$input = json_decode(file_get_contents('php://input'), true);

if (empty($input['email']) || empty($input['password'])) {
http_response_code(400);
echo json_encode([
'status' => false,
'message' => 'Email and password are required'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$user = $this->User_model->get_by_email(strtolower($input['email']));

if (!$user) {
http_response_code(401);
echo json_encode([
'status' => false,
'message' => 'Invalid credentials'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

if (!password_verify($input['password'], $user->password)) {
http_response_code(401);
echo json_encode([
'status' => false,
'message' => 'Invalid credentials'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$token = $this->jwt->create([
'id' => $user->id,
'name' => $user->name,
'email' => $user->email
]);

http_response_code(200);
```



```

echo json_encode([
    'status' => true,
    'message' => 'Login successful',
    'data' => [
        'id' => $user->id,
        'name' => $user->name,
        'email' => $user->email
    ],
    'access_token' => $token,
    'token_type' => 'Bearer'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
    http_response_code(500);
    echo json_encode([
        'status' => false,
        'message' => 'Server error: ' . $e->getMessage()
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}
}

public function logout()
{
    header('Content-Type: application/json; charset=utf-8');
    ob_clean();

    try {
        $token = $this->jwt->get_token_from_request();

        if (!$token) {
            http_response_code(401);
            echo json_encode([
                'status' => false,
                'message' => 'No token provided'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $decoded = $this->jwt->verify($token);

        if (!$decoded) {

```

```
http_response_code(401);
echo json_encode([
    'status' => false,
    'message' => 'Invalid token'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$user = $this->User_model->get_by_id($decoded->id);

if (!$user) {
    http_response_code(404);
    echo json_encode([
        'status' => false,
        'message' => 'User not found'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

http_response_code(200);
echo json_encode([
    'status' => true,
    'message' => 'Logout successful'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
    http_response_code(500);
    echo json_encode([
        'status' => false,
        'message' => 'Server error: ' . $e->getMessage()
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}
}

public function me()
{
    header('Content-Type: application/json; charset=utf-8');
    ob_clean();

    try {
        $token = $this->jwt->get_token_from_request();
```

```
if (!$token) {
    http_response_code(401);
    echo json_encode([
        'status' => false,
        'message' => 'No token provided'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

$decoded = $this->jwt->verify($token);

if (!$decoded) {
    http_response_code(401);
    echo json_encode([
        'status' => false,
        'message' => 'Invalid or expired token'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

$user = $this->User_model->get_by_id($decoded->id);

if (!$user) {
    http_response_code(404);
    echo json_encode([
        'status' => false,
        'message' => 'User not found'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

http_response_code(200);
echo json_encode([
    'status' => true,
    'message' => 'User retrieved successfully',
    'data' => [
        'id' => $user->id,
        'name' => $user->name,
        'email' => $user->email,
        'created_at' => $user->created_at,
        'updated_at' => $user->updated_at
    ]
]);
```

```

], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
http_response_code(500);
echo json_encode([
'status' => false,
'message' => 'Server error: ' . $e->getMessage()
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}
}
}

```

Post.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Post extends MX_Controller {

public function __construct()
{
parent::__construct();
$this->load->model('Post_model');
$this->load->library('jwt');
$this->load->library('upload');
}

private function parse_multipart_form()
{
$content_type = $_SERVER['CONTENT_TYPE'] ?? "";
$input = [];

if (strpos($content_type, 'multipart/form-data') !== false) {
$boundary = "";
if (preg_match('/boundary=([^\;]+)/', $content_type, $matches)) {
$boundary = trim($matches[1], "");
}

if (!$boundary) {
return [];
}
}

```

```

}

$raw = file_get_contents('php://input');
$parts = preg_split('/' . preg_quote("--$boundary") . '/', $raw);

foreach ($parts as $part) {
    if (empty(trim($part)) || $part === "--\r\n" || $part === "--") {
        continue;
    }

    $split = preg_split("/\r\n\r\n/", trim($part), 2);
    if (count($split) !== 2) {
        continue;
    }

    $headers = $split[0];
    $content = rtrim($split[1], "\r\n");

    if (preg_match('/name="([^\"]+)/', $headers, $matches)) {
        $name = $matches[1];

        if (preg_match('/filename="([^\"]+)/', $headers, $filename_matches)) {
            $file_content_type = 'application/octet-stream';
        }
        if (preg_match('/Content-Type:\s*([^\r\n]+)/', $headers, $type_matches)) {
            $file_content_type = trim($type_matches[1]);
        }

        $input[$name] = [
            'name' => $filename_matches[1],
            'type' => $file_content_type,
            'content' => $content
        ];
    } else {
        $input[$name] = $content;
    }
}

return $input;
}

if (strpos($content_type, 'application/x-www-form-urlencoded') !== false) {
    parse_str(file_get_contents('php://input'), $input);
}

```

```

return $input ?? [];
}

return array_merge($_POST ?? [], $_FILES ?? []);
}

private function get_image_url($image_filename)
{
if (empty($image_filename)) {
return null;
}
return 'http://localhost/ci3_project/uploads/posts/' . $image_filename;
}

public function handle($id = null)
{
$method = $_SERVER['REQUEST_METHOD'];

if ($method === 'GET') {
if ($id) {
$this->show($id);
} else {
$this->index();
}
} elseif ($method === 'POST') {
$this->store();
} elseif ($method === 'PUT') {
$this->update($id);
} elseif ($method === 'DELETE') {
$this->delete($id);
} else {
http_response_code(405);
header('Content-Type: application/json; charset=utf-8');
echo json_encode([
'status' => false,
'message' => 'Method not allowed'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}
}

public function index()
{
header('Content-Type: application/json; charset=utf-8');

```

```

ob_clean();

try {
$page = isset($_GET['page']) ? max(1, (int)$_GET['page']) : 1;
$per_page = isset($_GET['per_page']) ? max(1, min(100, (int)$_GET['per_page'])) : 10;
$offset = ($page - 1) * $per_page;

$posts = $this->Post_model->get_all($per_page, $offset);
$total = $this->Post_model->count_all();

foreach ($posts as &$post) {
if (!empty($post->image)) {
$post->image_url = $this->get_image_url($post->image);
} else {
$post->image_url = null;
}
}

http_response_code(200);
echo json_encode([
'status' => true,
'message' => 'Posts retrieved successfully',
'data' => $posts,
'pagination' => [
'total' => (int)$total,
'per_page' => $per_page,
'current_page' => $page,
'last_page' => ceil($total / $per_page)
]
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
http_response_code(500);
echo json_encode([
'status' => false,
'message' => 'Server error: ' . $e->getMessage()
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}
}

public function show($id)

```

```
{
header('Content-Type: application/json; charset=utf-8');
ob_clean();

try {
if (!is_numeric($id)) {
http_response_code(400);
echo json_encode([
'status' => false,
'message' => 'Invalid post ID'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$post = $this->Post_model->get_by_id($id);

if (!$post) {
http_response_code(404);
echo json_encode([
'status' => false,
'message' => 'Post not found'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

if (!empty($post->image)) {
$post->image_url = $this->get_image_url($post->image);
} else {
$post->image_url = null;
}

http_response_code(200);
echo json_encode([
'status' => true,
'message' => 'Post retrieved successfully',
'data' => $post
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
http_response_code(500);
echo json_encode([
'status' => false,
```



```

'message' => 'Server error: ' . $e->getMessage()
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}
}

public function store()
{
    header('Content-Type: application/json; charset=utf-8');
    ob_clean();

    try {
        $token = $this->jwt->get_token_from_request();

        if (!$token) {
            http_response_code(401);
            echo json_encode([
                'status' => false,
                'message' => 'Unauthorized: No token provided'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $decoded = $this->jwt->verify($token);

        if (!$decoded) {
            http_response_code(401);
            echo json_encode([
                'status' => false,
                'message' => 'Unauthorized: Invalid or expired token'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $input = json_decode(file_get_contents('php://input'), true);

        if (!$input && !empty($_POST)) {
            $input = $_POST;
        }

        if (empty($input['title']) || empty($input['article'])) {
            http_response_code(400);
            echo json_encode([

```

```
'status' => false,
'message' => 'Title and article are required'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$post_data = [
'title' => htmlspecialchars($input['title']),
'author' => htmlspecialchars($input['author'] ?? 'Anonymous'),
'article' => htmlspecialchars($input['article'])
];

$image_name = '';
if (!empty($_FILES['image']['name'])) {
$image_name = $this->upload_image();
if (!$image_name) {
http_response_code(400);
echo json_encode([
'status' => false,
'message' => 'Image upload failed'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}
$post_data['image'] = $image_name;
}

$post_id = $this->Post_model->create($post_data);

if (!$post_id) {
http_response_code(500);
echo json_encode([
'status' => false,
'message' => 'Failed to create post'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$post = $this->Post_model->get_by_id($post_id);

if (!empty($post->image)) {
$post->image_url = $this->get_image_url($post->image);
} else {
$post->image_url = null;
}
```

```

}

http_response_code(201);
echo json_encode([
    'status' => true,
    'message' => 'Post created successfully',
    'data' => $post
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
    http_response_code(500);
    echo json_encode([
        'status' => false,
        'message' => 'Server error: ' . $e->getMessage()
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}
}

public function update($id)
{
    header('Content-Type: application/json; charset=utf-8');
    ob_clean();

    try {
        $token = $this->jwt->get_token_from_request();

        if (!$token) {
            http_response_code(401);
            echo json_encode([
                'status' => false,
                'message' => 'Unauthorized: No token provided'
            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
            exit;
        }

        $decoded = $this->jwt->verify($token);

        if (!$decoded) {
            http_response_code(401);
            echo json_encode([
                'status' => false,

```

```
'message' => 'Unauthorized: Invalid or expired token'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

if (!is_numeric($id)) {
    http_response_code(400);
    echo json_encode([
        'status' => false,
        'message' => 'Invalid post ID'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

if (!$this->Post_model->exists($id)) {
    http_response_code(404);
    echo json_encode([
        'status' => false,
        'message' => 'Post not found'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

$input = json_decode(file_get_contents('php://input'), true);
if (!$input) {
    $input = $this->parse_multipart_form();
}

if (is_array($input['image']) && isset($input['image']['name']) &&
    isset($input['image']['content'])) {
    $_FILES['image'] = [
        'name' => $input['image']['name'] ?? 'image_' . time() . '.png',
        'type' => $input['image']['type'] ?? 'image/png',
        'tmp_name' => $this->_save_temp_file($input['image']['content']),
        'error' => 0,
        'size' => strlen($input['image']['content'])
    ];
}

if (empty($input['title']) || empty($input['article'])) {
    http_response_code(400);
    echo json_encode([
        'status' => false,
        'message' => 'Title and article are required'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}
```

```

], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$post_data = [
'title' => htmlspecialchars($input['title']),
'author' => htmlspecialchars($input['author'] ?? 'Anonymous'),
'article' => htmlspecialchars($input['article'])
];

if ((!empty($_FILES['image']['name'])) || (isset($input['image']) &&
is_array($input['image']) && isset($input['image']['name']))) {
$sold_post = $this->Post_model->get_by_id($id);

if (!empty($sold_post->image)) {
$sold_file = FCPATH . 'uploads/posts/' . $sold_post->image;
if (file_exists($sold_file)) {
unlink($sold_file);
}
}

$image_name = $this->_upload_image();
if ($image_name) {
$post_data['image'] = $image_name;
}
// If image upload fails, just skip it - don't fail the whole request for PUT
}

$result = $this->Post_model->update($id, $post_data);

if (!$result) {
http_response_code(500);
echo json_encode([
'status' => false,
'message' => 'Failed to update post'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$post = $this->Post_model->get_by_id($id);

if (!empty($post->image)) {
$post->image_url = $this->get_image_url($post->image);
}

```

```
} else {
$post->image_url = null;
}

http_response_code(200);
echo json_encode([
'status' => true,
'message' => 'Post updated successfully',
'data' => $post
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
http_response_code(500);
echo json_encode([
'status' => false,
'message' => 'Server error: ' . $e->getMessage()
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}
}

public function delete($id)
{
header('Content-Type: application/json; charset=utf-8');
ob_clean();

try {
$token = $this->jwt->get_token_from_request();

if (!$token) {
http_response_code(401);
echo json_encode([
'status' => false,
'message' => 'Unauthorized: No token provided'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$decoded = $this->jwt->verify($token);

if (!$decoded) {
http_response_code(401);
```

```
echo json_encode([
'status' => false,
'message' => 'Unauthorized: Invalid or expired token'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

if (!is_numeric($id)) {
http_response_code(400);
echo json_encode([
'status' => false,
'message' => 'Invalid post ID'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

$post = $this->Post_model->get_by_id($id);

if (!$post) {
http_response_code(404);
echo json_encode([
'status' => false,
'message' => 'Post not found'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}

if (!empty($post->image)) {
$file = FCPATH . 'uploads/posts/' . $post->image;
if (file_exists($file)) {
unlink($file);
}
}

$result = $this->Post_model->delete($id);

if (!$result) {
http_response_code(500);
echo json_encode([
'status' => false,
'message' => 'Failed to delete post'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}
```

```

}

http_response_code(200);
echo json_encode([
'status' => true,
'message' => 'Post deleted successfully'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
http_response_code(500);
echo json_encode([
'status' => false,
'message' => 'Server error: ' . $e->getMessage()
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;
}
}

private function _save_temp_file($content)
{
$temp_path = sys_get_temp_dir() . '/ci3_upload_' . uniqid() . '.tmp';
file_put_contents($temp_path, $content);
return $temp_path;
}

private function _upload_image()
{
$config['upload_path'] = FCPATH . 'uploads/posts/';
$config['allowed_types'] = 'gif|jpg|png|jpeg';
$config['max_size'] = 2048;
$config['file_name'] = 'post_' . time() . '_' . uniqid();

if (!is_dir($config['upload_path'])) {
mkdir($config['upload_path'], 0755, true);
}

$this->upload->initialize($config);

if (!$this->upload->do_upload('image')) {
return false;
}

```



```
$upload_data = $this->upload->data();  
return $upload_data['file_name'];  
}  
}
```

5. Analisis & Pembahasan

Pada praktikum ini, penulis berhasil mengimplementasikan restfullAPI menggunakan CodeIgniter 3. Seluruh fitur utama seperti Register, Login dengan Bearer Token, Logout, serta operasi CRUD Post (termasuk upload gambar) dapat berjalan dengan baik. Beberapa kendala teknis seperti error view folder path dan HTTP Error 500 berhasil diatasi melalui konfigurasi yang tepat pada file index.php, config.php, dan database.php. Pengujian menggunakan Postman menunjukkan bahwa semua endpoint merespons sesuai harapan dengan status code yang benar. Proyek ini secara keseluruhan berhasil dibuat dan diuji.

6. Kesimpulan

Praktikum ini berhasil dilaksanakan dengan baik. Penulis telah berhasil membuat restfullAPI Authentication dan CRUD Post menggunakan Framework CodeIgniter 3 beserta fitur upload gambar. Melalui praktikum ini, penulis semakin memahami konsep pembuatan API, autentikasi, dan penanganan error pada CodeIgniter. Hasil ini dapat menjadi dasar untuk pengembangan aplikasi web atau mobile di masa mendatang.

7. Lampiran

Link repositori GitHub

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/FutuwahlqamatulHaqi-707012500099>